scaled version of) the $\mathcal{X}^2$ distribution converges to the F-distribution.

$$\frac{1}{p^{\text{large}} - p^{\text{small}}} \mathcal{X}^2_{\text{df} = p^{\text{large}} - p^{\text{small}}} \overset{d}{\approx} F_{(p^{\text{large}} - p^{\text{small}}),\, (n - p^{\text{large}})}$$

However, in practice we should always use the exact F-test for linear regression.

- Since $\mathcal{X}^2$ likelihood ratio test LRT is only applicable to nested model selection, in order to compare two non-nested models, we should use Akaike Information Criterion AIC or Bayes Information Criterion BIC instead (similar to the approach for linear regression).

```
# Compute AIC
AIC(model)
model$aic
-2 * logLik(model) + 2 * p.large
```

```
[1] 59.24394
[1] 59.24394
'log Lik.' 59.24394 (df=5)
```

- Now we understand all the numbers reported in R's `summary()` for logistic regression model.

```
summary(model)
```

```

Call:
glm(formula = y ~ x1 + x2 + x3 + x4, family = binomial, data = data)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)   0.5568     0.4315   1.290 0.196911
x1            2.3817     0.6238   3.818 0.000135 ***
x2            4.5772     1.1632   3.935 8.32e-05 ***
x3            0.3147     0.3652   0.862 0.388886
x4            0.9580     0.5423   1.766 0.077317 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 137.989  on 99  degrees of freedom
Residual deviance:  49.244  on 95  degrees of freedom
AIC: 59.244

Number of Fisher Scoring iterations: 7
```

7 Poisson Regression

7.1 Poisson(λ)

- A random variable $X \sim \text{Poisson}(\lambda)$ with **rate** or **intensity** parameter $\lambda > 0$ takes on value in the non-negative integers $\{0, 1, 2, \ldots\}$.

```
# Simulate Poisson(4) observations
lambda <- 4
rpois(20, lambda = lambda)
```

```
[1] 6 5 3 2 2 1 1 2 1 1 4 6 5 1 0 4 2 5 5 6
```

- Probability mass function p_X is

$$P(X = x) = p_X(x) = e^{-\lambda}\frac{\lambda^x}{x!} \quad \text{for } x = 0, 1, 2, \ldots$$

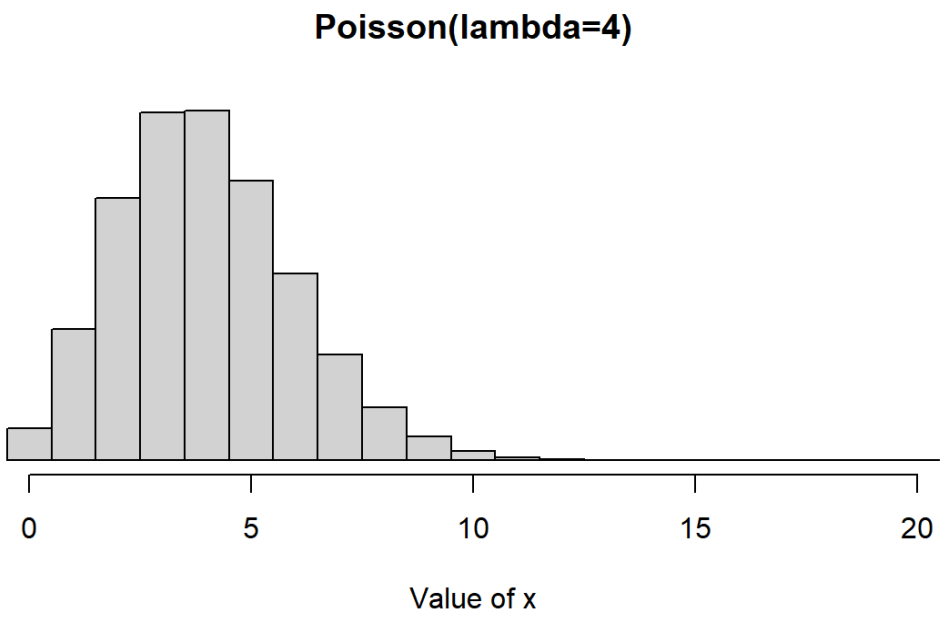
  where $e = 2.71828\ldots$ is the Euler's number (https://en.wikipedia.org/wiki/E_(mathematical_constant)) for natural logarithm (https://en.wikipedia.org/wiki/Natural_logarithm). The power series (https://en.wikipedia.org/wiki/Power_series)

$$1 + \lambda + \frac{\lambda^2}{2!} + \frac{\lambda^3}{3!} + \cdots = \sum_{x=1}^{\infty} \frac{\lambda^x}{x!} = e^{\lambda}$$

  sums up accordingly.
- Mean $\mathbb{E}(X)$ and variance $\mathrm{Var}(X)$ are the same i.e.

$$\mathbb{E}(X) = \mathrm{Var}(X) = \lambda$$

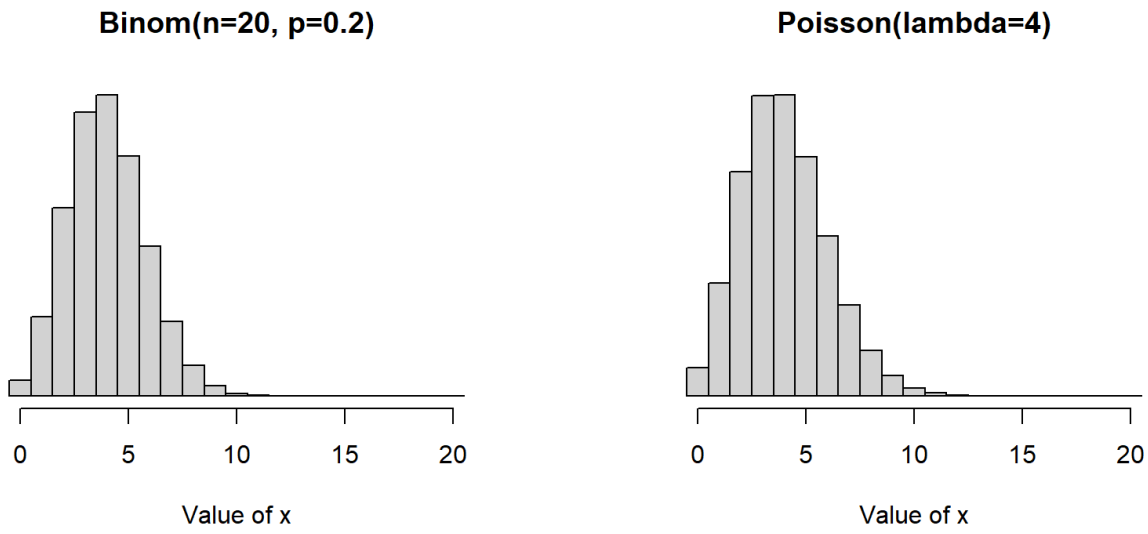
- Probability mass function p_X is unimodal, centered near its mean $E(X) = \lambda$, and right-skewed.



- In practice, it is often used in a generalized linear model (https://en.wikipedia.org/wiki/Generalized_linear_model) GLM called Poisson regression (https://en.wikipedia.org/wiki/Poisson_regression) to count the number of occurrences of a large number of possible random events, each of which is rare. For example, the number of pedestrians in a city involved in traffic accidents in a year.
- **Poisson limit theorem** (https://en.wikipedia.org/wiki/Poisson_limit_theorem) / law of rare events (https://en.wikipedia.org/wiki/Poisson_distribution#Law_of_rare_events): If n is large and $p \approx 0$ is small, then

$$\mathrm{Binom}(n, p) \overset{d}{\approx} \mathrm{Poisson}(\lambda = np)$$

  In other words, in this case, we could simplify our modeling and computation by approximating the exact binomial distribution with a related Poisson distribution.



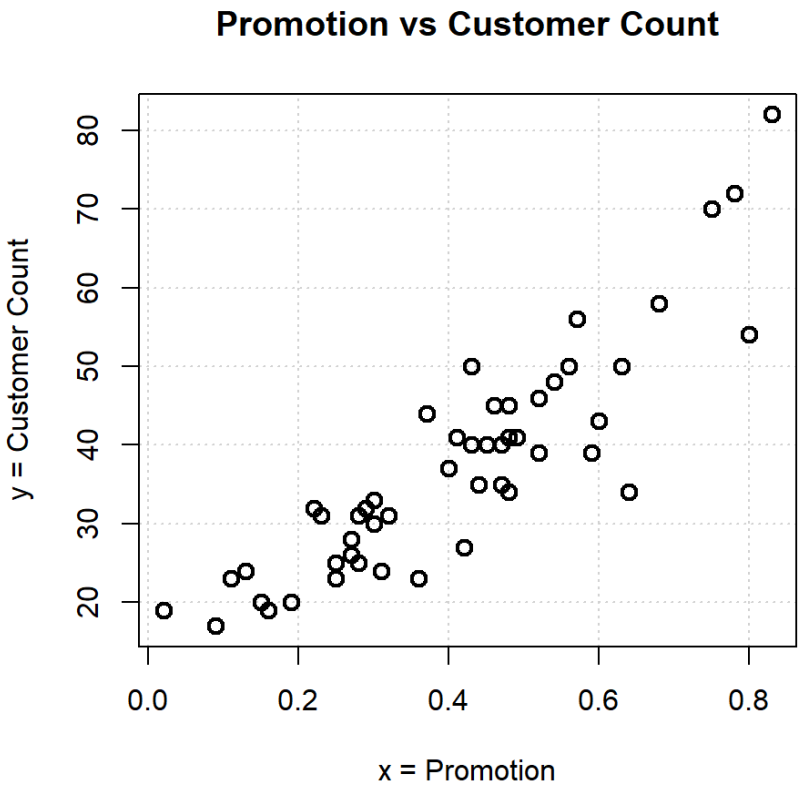
7.2 Count Data

- **Poisson regression** (https://en.wikipedia.org/wiki/Poisson_regression) is a kind of generalized linear model GLM for count data i.e. the response variable $Y \in \{0, 1, 2, \dots, \}$ takes non-negative integer value only.
- *Example:* Let Y be the number of cars in a city that had an accident in a year.
  - Strictly speaking, if we assume there are only n number of cars in the city, then $Y \leq n$ is bounded above. So it is not exactly a Poisson distribution (whose value is unbounded).
  - Moreover, if we assume each car has an independent probability $p \in (0, 1)$ to be involved in an accident, then Y follows a $\text{Binom}(n, p)$ binomial distribution. So it should be modeled using logistic regression instead.
  - Nonetheless, if $n \gg 1$ is large and $p \approx 0$ is small, then the Poisson limit theorem says

    $$Y \sim \text{Binom}(n, p) \overset{d}{\approx} \text{Poisson}(\lambda = np)$$

    is approximately Poisson distributed. Therefore, Poisson regression model is sometime used as an approximation for simplicity.
- *Example:* Let Y be the number of customers that arrive at a restaurant within a fixed period of time e.g. an hour or a day. Let x be the amount of promotion discount during that period of time.
  - In statistics literature, such an arrival process (https://en.wikipedia.org/wiki/Markovian_arrival_process) might be modeled as a Poisson point process (https://en.wikipedia.org/wiki/Poisson_process) with exponentially distributed (https://en.wikipedia.org/wiki/Exponential_distribution) (random) inter-arrival time. In this case, the number of arrivals itself follows a Poisson distribution.

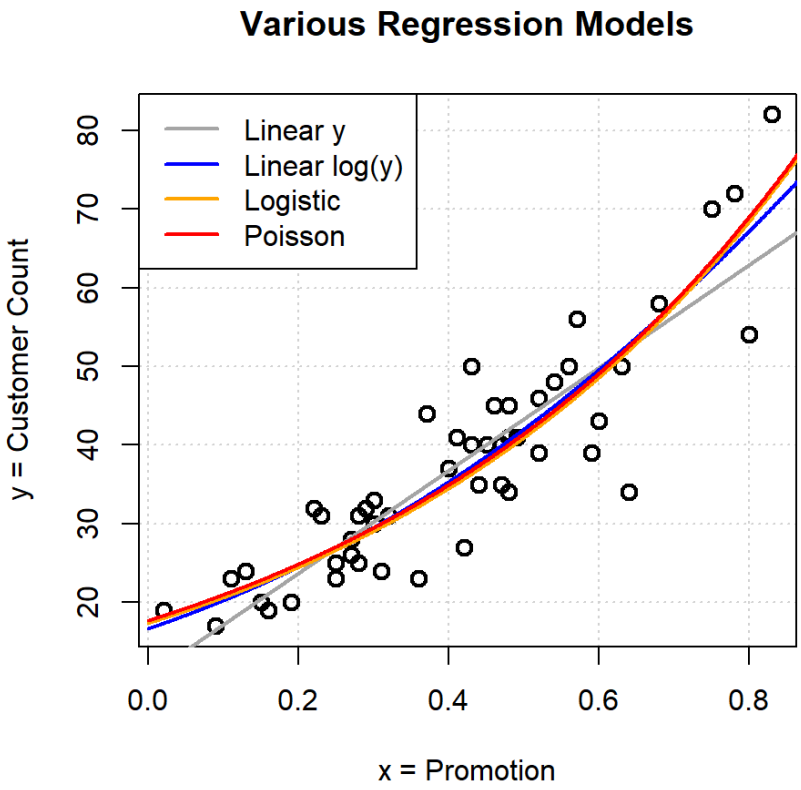
```
promo <- c(0.02, 0.09, 0.11, 0.13, 0.15, 0.16, 0.19, 0.22, 0.23, 0.25,
           0.25, 0.27, 0.27, 0.28, 0.28, 0.29, 0.30, 0.30, 0.31, 0.32,
           0.36, 0.37, 0.40, 0.41, 0.42, 0.43, 0.43, 0.44, 0.45, 0.46,
           0.47, 0.47, 0.48, 0.48, 0.48, 0.49, 0.52, 0.52, 0.54, 0.56,
           0.57, 0.59, 0.60, 0.63, 0.64, 0.68, 0.75, 0.78, 0.80, 0.83)
customer <- c(19, 17, 23, 24, 20, 19, 20, 32, 31, 25,
              23, 28, 26, 25, 31, 32, 33, 30, 24, 31,
              23, 44, 37, 41, 27, 50, 40, 35, 40, 45,
              40, 35, 45, 41, 34, 41, 46, 39, 48, 50,
              56, 39, 43, 50, 34, 58, 70, 72, 54, 82)
```



- Sometimes we view a large number n of occurrences of events as approximately normally distributed too. This is based on the central limit theorem CLT (https://en.wikipedia.org/wiki/Central_limit_theorem).

$$Y \sim \text{Binom}(n, p) \overset{d}{\approx} \mathcal{N}(\mu = np, \sigma^2 = np(1-p)) \quad \text{for large } n \gg 1$$

- In practice, a data set could be viewed in multiple ways and modeled accordingly e.g. using linear regression / logistic regression / Poisson regression.




```
# Model 1: Linear regression with y
model.1 <- lm(customer ~ promo)

summary(model.1)
```

```

Call:
lm(formula = customer ~ promo)

Residuals:
     Min       1Q   Median       3Q      Max
-18.5004  -3.7695  -0.1934   3.3779  17.0908

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   10.702      2.239   4.779  1.7e-05 ***
promo         65.310      4.962  13.162  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 6.661 on 48 degrees of freedom
Multiple R-squared:  0.783, Adjusted R-squared:  0.7785
F-statistic: 173.2 on 1 and 48 DF,  p-value: < 2.2e-16
```

```
# Model 2: Linear regression with log(y)
model.2 <- lm(log(customer) ~ promo)

summary(model.2)
```

```
Call:
lm(formula = log(customer) ~ promo)

Residuals:
     Min       1Q   Median       3Q      Max
-0.42573 -0.11639  0.03381  0.09984  0.31989

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  2.85509    0.05438   52.50   <2e-16 ***
promo        1.71406    0.12051   14.22   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.1618 on 48 degrees of freedom
Multiple R-squared:  0.8082,    Adjusted R-squared:  0.8042
F-statistic: 202.3 on 1 and 48 DF,  p-value: < 2.2e-16
```

```
# Model 3: Logistic regression with group size assumption
total.ppl <- 200 # Group size n_i
k <- length(promo) # Number of groups k
model.3 <- glm(customer / total.ppl ~ promo, family = binomial, weights = rep(total.ppl, k))

summary(model.3)
pchisq(deviance(model.3), df.residual(model.3), lower.tail = FALSE) # P-value of X^2 goodness-of-fit test
```

```
Call:
glm(formula = customer/total.ppl ~ promo, family = binomial,
    weights = rep(total.ppl, k))

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -2.39786    0.06764  -35.45   <2e-16 ***
promo        2.14573    0.13690   15.67   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 309.634  on 49  degrees of freedom
Residual deviance:  58.246  on 48  degrees of freedom
AIC: 321.68

Number of Fisher Scoring iterations: 4

[1] 0.1476883
```

```
# Model 4: Poisson regression
model.4 <- glm(customer ~ promo, family = poisson)

summary(model.4)
pchisq(deviance(model.4), df.residual(model.4), lower.tail = FALSE) # P-value of X^2 goodness-of-fit test
```

```
Call:
glm(formula = customer ~ promo, family = poisson)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  2.87376    0.06087   47.21   <2e-16 ***
promo        1.69984    0.11923   14.26   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 247.151  on 49  degrees of freedom
Residual deviance:  45.697  on 48  degrees of freedom
AIC: 319.69

Number of Fisher Scoring iterations: 4

[1] 0.5676976
```

7.3 Log Link Function

- Let $(\vec{x}_1, y_1), \ldots, (\vec{x}_n, y_n)$ be n pairs of observed data points with d-dimensional explanatory variable $\vec{x}_i = (x_{i1}, \ldots, x_{id}) \in \mathbb{R}^d$ and count response variable $y_i \in \{0, 1, 2, \ldots\}$.
- The statistical model assumption of Poisson regression is:
    1. **Deterministic explanatory variable:** Explanatory variables $\vec{x}_i$ is fixed / deterministic / not a random variable in this statistical model.
    2. **Random response variable:** Response variable Y_i is viewed as a random variable. Each observed response value y_i is a realization of its random variable Y_i.
    3. **Independence:** The response variables Y_i's are independent random variables. (In this case, the likelihood function factors into a product of n terms for easier mathematical computation later.)
    4. **Poisson distributed:** Since each random variable $Y_i \in \{0, 1, 2, \ldots\}$ is a non-negative integer, we assume a Poisson distribution

    $$Y_i \sim \text{Poisson}(\lambda_i)$$

    For each ith observed data point, the Poisson intensity parameter $\lambda_i > 0$ can be different.
    5. **Linear:** There is an increasing relationship between the expected count $\lambda_i = \mathbb{E}(Y_i)$ and linear combination of explanatory variable $\vec{x}_i$.

    $$\underbrace{\mathbb{E}(Y_i)}_{\lambda_i} \quad \text{vs} \quad \underbrace{\beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id}}_{\text{linear combination}}$$

    If the right-hand side increases, then the left-hand side should increase too, and vice versa. Here the $\beta_0, \ldots, \beta_d$ parameters are common across all n data points. They are unknown and are to be estimated from the data.
    6. **Log as link function:** There is some monotonic increasing function $f : \mathbb{R} \mapsto (0, \infty)$ that links the right-hand side and left-hand side together.

    $$\underbrace{\mathbb{E}(Y_i)}_{\lambda_i} = f(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id})$$

    For each ith observed data point, since $\vec{x}_i$ can be different, so could λ_i. There is actually a natural / default / canonical choice for this function f i.e. the exponential function $f(x) = e^x$. Combining the above together, the

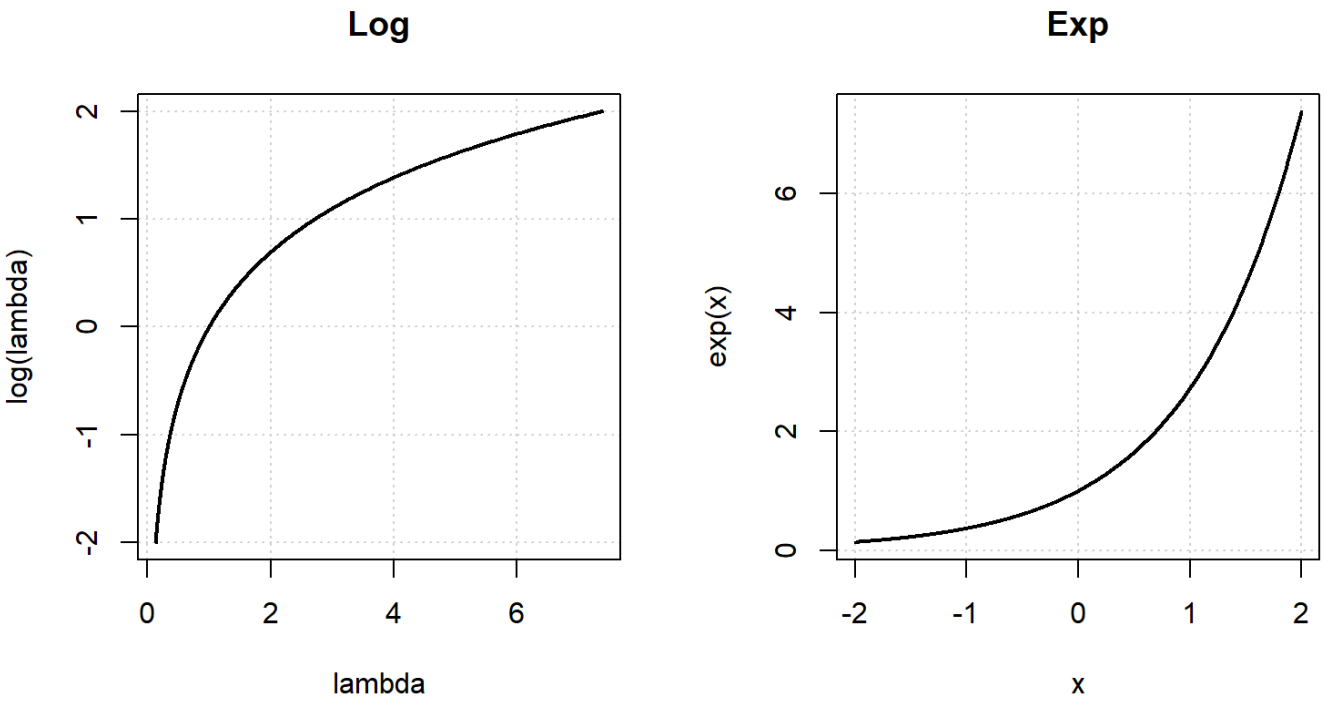
Poisson regression model equation is

$$\mathbb{E}(Y_i) = \lambda_i = e^{\beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id}}$$

Equivalently, it is

$$\log \mathbb{E}(Y_i) = \log \lambda_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id}$$

(Technically, in statistics literature, our function f is not referred to as the link function. Instead, its inverse function $f^{-1} = \log$ is the link function.)



- Although the Poisson model equation above looks similar to a linear regression model when the response variable Y_i is under the logarithm transformation $\log(Y_i)$ i.e. R's `lm(log(y) ~ ...)`, these two models are different.
  - The linear regression model is

$$\mathbb{E}(\log Y_i) = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id}$$

  - By Jensen's inequality (https://en.wikipedia.org/wiki/Jensen%27s_inequality), we have

$$\mathbb{E}(\log Y_i) < \log \mathbb{E}(Y_i)$$

    So they are not the same quantity.
  - More importantly, Poisson regression is $Y_i \sim \text{Poisson}(\lambda_i)$. But this linear regression is $\log Y_i \sim \mathcal{N}(\mu_i, \sigma^2)$ i.e. Y_i itself has a log-normal (https://en.wikipedia.org/wiki/Log-normal_distribution) distribution.
- Overall, Poisson regression with log link function is the statistical model

$$Y_i \sim \text{Poisson}\underbrace{\left(e^{\beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id}}\right)}_{\lambda_i > 0} \quad \text{for } i = 1, \ldots, n$$

  with parameters $\beta_0, \ldots, \beta_d$.
- Interpretation of coefficient β_j: If x_j increases by $+1$ unit to become $x_j + 1$ while all other explanatory variables do not change, then the expected count become

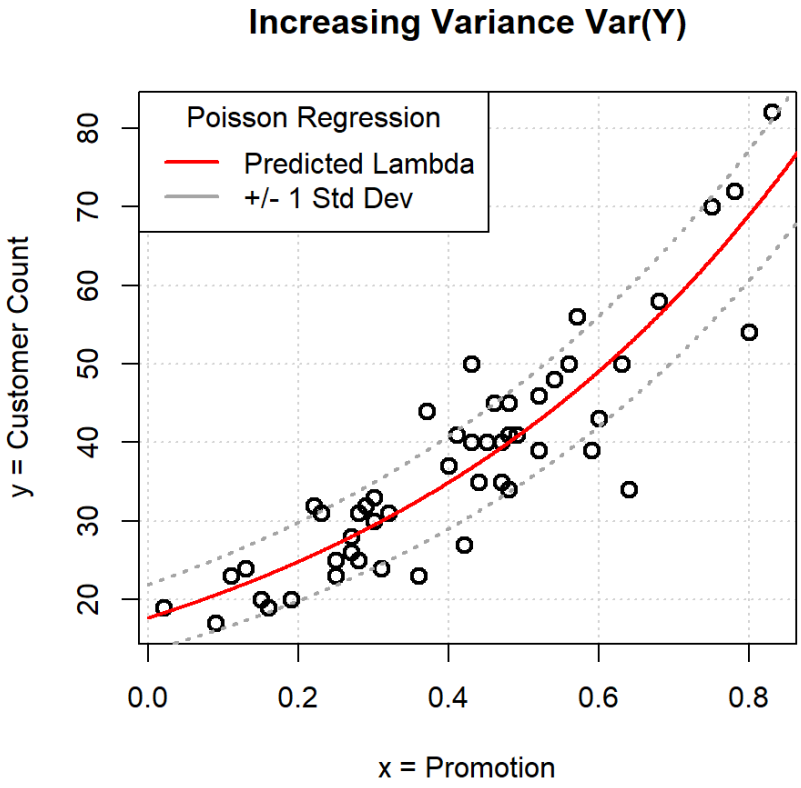
$$\mathbb{E}(Y_i^{\text{new}}) = \lambda_i^{\text{new}} = e^{\beta_0+\beta_1 x_{i1}+\cdots+\beta_j(x_{ij}+1)+\cdots\beta_d x_{id}} = \lambda_i^{\text{old}} \times e^{\beta_j}$$

The effect is multiplicative.

- According to properties of the Poisson distribution, we have

$$\text{Var}(Y_i) = \mathbb{E}(Y_i) = \lambda_i$$

Therefore, the variance $\text{Var}(Y_i)$ of the count response variable Y_i increases naturally as the Poisson intensity parameter λ_i / magnitude of Y_i becomes larger.



7.4 Model Estimation

- Given n pairs of observed data points $(\vec{x}_1, y_1), \ldots, (\vec{x}_n, y_n)$, we follow the maximum likelihood principle to estimate the $\beta_0, \ldots, \beta_d$ parameters. Recall the likelihood function for Poisson distribution is

$$L(\beta_0, \ldots, \beta_d) = \prod_{i=1}^{n} P(Y_i = y_i) = \prod_{i=1}^{n} e^{-\lambda_i} \frac{\lambda_i^{y_i}}{y_i!} \quad \text{with} \quad \lambda_i = e^{\beta_0+\beta_1 x_{i1}+\cdots+\beta_d x_{id}}$$

Therefore, $L(\beta_0, \ldots, \beta_d)$ is indeed a function of the $\beta_0, \ldots, \beta_d$ parameters. Now the log-likelihood function is

$$l(\beta_0, \ldots, \beta_d) = \log L(\beta_0, \ldots, \beta_d) = \sum_{i=1}^{n} -\lambda_i + y_i \log \lambda_i - \log(y_i!)$$

The maximum likelihood estimate MLE is

$$\beta_0^*, \ldots, \beta_d^* = \underset{\beta_0,\ldots,\beta_d}{\arg\max}\, l(\beta_0, \ldots, \beta_d) = \underset{\beta_0,\ldots,\beta_d}{\arg\max} \sum_{i=1}^{n} -\lambda_i + y_i \log \lambda_i - \underbrace{\log(y_i!)}_{\text{constant}}$$

- Similarly to logistic regression, this MLE optimization problem does not have any closed-form solution. It is solved using some iterative algorithm such as Fisher (https://en.wikipedia.org/wiki/Fisher_information) scoring

(https://en.wikipedia.org/wiki/Scoring_algorithm) algorithm.

- In R, use the `glm(..., family=poisson(...))` function to fit a Poisson regression model.

```
promo <- c(0.02, 0.09, 0.11, 0.13, 0.15, 0.16, 0.19, 0.22, 0.23, 0.25,
           0.25, 0.27, 0.27, 0.28, 0.28, 0.29, 0.30, 0.30, 0.31, 0.32,
           0.36, 0.37, 0.40, 0.41, 0.42, 0.43, 0.43, 0.44, 0.45, 0.46,
           0.47, 0.47, 0.48, 0.48, 0.48, 0.49, 0.52, 0.52, 0.54, 0.56,
           0.57, 0.59, 0.60, 0.63, 0.64, 0.68, 0.75, 0.78, 0.80, 0.83)
customer <- c(19, 17, 23, 24, 20, 19, 20, 32, 31, 25,
              23, 28, 26, 25, 31, 32, 33, 30, 24, 31,
              23, 44, 37, 41, 27, 50, 40, 35, 40, 45,
              40, 35, 45, 41, 34, 41, 46, 39, 48, 50,
              56, 39, 43, 50, 34, 58, 70, 72, 54, 82)

model <- glm(customer ~ promo, family = poisson) # Log link is default. See ?family

# Link function
model$family$link # See str(model)

# Beta's
model
coef(model) # Same as model$coefficients and coefficients(model)
summary(model)
```

```
[1] "log"
```

```

Call:  glm(formula = customer ~ promo, family = poisson)

Coefficients:
(Intercept)        promo
      2.874        1.700

Degrees of Freedom: 49 Total (i.e. Null);  48 Residual
Null Deviance:      247.2
Residual Deviance: 45.7     AIC: 319.7
```

```
(Intercept)       promo
   2.873761    1.699845
```

```
Call:
glm(formula = customer ~ promo, family = poisson)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  2.87376    0.06087   47.21   <2e-16 ***
promo        1.69984    0.11923   14.26   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 247.151  on 49  degrees of freedom
Residual deviance:  45.697  on 48  degrees of freedom
AIC: 319.69

Number of Fisher Scoring iterations: 4
```

```
# Fitted values / predictions (for lambda_i)
fitted(model) # Same as fitted.values(model) and model$fitted.values
```

```
       1        2        3        4        5        6
18.31568 20.63001 21.34342 22.08151 22.84512 23.23677
```

```
# Fitted linear predictor i.e. b0 + b1*x1 + ... + b_d*x_d before link function
b0 <- coef(model)[1]
b1 <- coef(model)[2]
b0 + b1 * promo

model$linear.predictors
```

```
[1] 2.907758 3.026747 3.060744 3.094740 3.128737 3.145736
       1        2        3        4        5        6
2.907758 3.026747 3.060744 3.094740 3.128737 3.145736
```

```
# Compute fitted intensity lambda_i after log link
exp(model$linear.predictors)
```

```
       1        2        3        4        5        6
18.31568 20.63001 21.34342 22.08151 22.84512 23.23677
```

```
# Predict linear predictor before link function
x.new <- 0.3

b0 + b1 * x.new
(u <- predict(model, newdata=data.frame(promo=x.new), type='link')) # b0 + b1*x1 + ... + b_d*x_d
```

```
(Intercept)
   3.383714
       1
3.383714
```

```
# Predict intensity lambda after log link
exp(u) # Compute log link
predict(model, newdata=data.frame(promo=x.new), type='response') # lambda
```

```
       1
29.48006
       1
29.48006
```

7.5 Deviance

- For Poisson regression, the concepts of null model / saturated model / null deviance / residual deviance / degrees of freedom / $\mathcal{X}^2$ goodness-of-fit test / Wald z-test / $\mathcal{X}^2$ likelihood ratio test LRT are the same as logistic regression. However, some of them require different formulae to compute.
- The log-likelihood value of the null model is

$$l_{\text{null}} = \sum_{i=1}^{n} -\bar{y} + y_i \log \bar{y} - \log(y_i!)$$

with constant intensity λ^{null} based on the overall mean count $\bar{y}$ i.e.

$$\lambda^{\text{null}} = \bar{y} = \frac{1}{n} \sum_{i=1}^{n} y_i$$

```
# Null model
null.model <- glm(customer ~ 1, family=poisson)

summary(null.model)
```

```

Call:
glm(formula = customer ~ 1, family = poisson)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  3.62274    0.02311   156.7   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 247.15  on 49  degrees of freedom
Residual deviance: 247.15  on 49  degrees of freedom
AIC: 519.14

Number of Fisher Scoring iterations: 4
```

```
# Predicted intensity for null model
(lambda.null <- fitted(null.model))

y <- customer
mean(y) # Overall mean
```

```
    1     2     3     4     5     6
37.44 37.44 37.44 37.44 37.44 37.44
[1] 37.44
```

```
# Log-likelihood of null model
logLik(null.model) # Or logLik(null.model)[1]
(l.null <- sum(-lambda.null + y * log(lambda.null) - lfactorial(y))) # See ?lfactorial
```

```
'log Lik.' -258.5701 (df=1)
[1] -258.5701
```

- Let

$$l_{\text{model}} = \sum_{i=1}^{n} -\hat{\lambda}_i + y_i \log \hat{\lambda}_i - \log(y_i!)$$

be the usual log-likelihood value of the fitted model of interest. For the ith data point y_i, this model predicts the usual fitted intensity $\hat{\lambda}_i$.

```
# Model of interest
model <- glm(customer ~ promo, family = poisson)

# Log-likelihood of model
logLik(model)

lambda.model <- fitted(model) # Usual predicted intensity lambda_i
(l.model <- sum(-lambda.model + y * log(lambda.model) - lfactorial(y)))
```

```
'log Lik.' -157.8429 (df=2)
[1] -157.8429
```

- Let

$$l_{\text{saturated}} = \sum_{i=1}^{n} -y_i + y_i \log y_i - \log(y_i!)$$

be the log-likelihood value of the saturated model with perfect intensity

$$\lambda_i^{\text{saturated}} = y_i \quad \text{for } i = 1, \ldots, n$$

based on maximum likelihood estimate MLE of Poisson distribution.

```
# Predicted intensity for saturated model
(lambda.saturated <- y)

# Log-likelihood of saturated model
(l.saturated <- sum(-lambda.saturated + y * log(lambda.saturated) - lfactorial(y)))
```

```
[1] 19 17 23 24 20 19
[1] -134.9943
```

- These three log-likelihood values continue to satisfy the following relationship with $l_{\text{saturated}}$ being the upper bound of log-likelihood value over all possible models.

$$l_{\text{null}} < l_{\text{model}} < l_{\text{saturated}} < 0$$

```
c(l.null, l.model, l.saturated)
```

```
[1] -258.5701 -157.8429 -134.9943
```

- Let's define **null deviance**

$$D_{\text{null}} = -2\left(l_{\text{null}} - l_{\text{saturated}}\right) > 0$$

and **residual deviance** (or simply "**deviance**")

$$\begin{aligned} D_{\text{residual}} &= -2\left(l_{\text{model}} - l_{\text{saturated}}\right) \\ &= \sum_{i=1}^{n} \underbrace{2\left(-(y_i - \hat{\lambda}_i) + y_i \log\left(y_i/\hat{\lambda}_i\right)\right)}_{\text{deviance residual ?}} \\ &= 2\sum_{i=1}^{n} y_i \log\left(y_i/\hat{\lambda}_i\right) \qquad \text{if model has intercept term} \end{aligned}$$

If $y_i = 0$, then we interpret $0 \times \log 0 = 0 \times -\infty = 0$. When the model has an intercept term, the last equality holds because

$$\sum_{i=1}^{n} y_i = \sum_{i=1}^{n} \hat{\lambda}_i$$

```
summary(model) # Model of interest
```

```

Call:
glm(formula = customer ~ promo, family = poisson)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  2.87376    0.06087   47.21   <2e-16 ***
promo        1.69984    0.11923   14.26   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 247.151  on 49  degrees of freedom
Residual deviance:  45.697  on 48  degrees of freedom
AIC: 319.69

Number of Fisher Scoring iterations: 4
```

```
# Null deviance
-2 * (l.null - l.saturated)
model$null.deviance
```

```
[1] 247.1514
[1] 247.1514
```

```
# Residual deviance
-2 * (l.model - l.saturated)
deviance(model) # Same as model$deviance

2 * sum(y * log(y / lambda.model)) # Because model has intercept term
```

```
[1] 45.69717
[1] 45.69717
[1] 45.69717
```

```
c(sum(y), sum(lambda.model)) # Same when model has intercept term
```

```
[1] 1872 1872
```

```
# Model without intercept term
model.b1 <- glm(customer ~ 0 + promo, family = poisson)

c(sum(y), sum(fitted(model.b1))) # Not necessarily the same
```

```
[1] 1872.000 1394.432
```

- $\mathcal{X}^2$ **goodness-of-fit test** using D_{residual} is the same as logistic regression. Here the asymptotic / regularity condition for the (somewhat) validity of the test requires each y_i to be sufficiently large e.g. $y_i \geq 10$. Similarly, **Wald** z**-test** using $\hat{\beta}_j / \text{s.e.}(\hat{\beta}_j)$ and $\mathcal{X}^2$ **likelihood ratio test LRT** using $D_{\text{residual}}^{\text{small}} - D_{\text{residual}}^{\text{large}}$ for model selection are also the same as logistic regression.

7.6 Residual

- Poisson regression has the same kinds of residual as logistic regression. However, some of them require different formulae to compute.
- **Response residual**: Simple difference between the observed count outcome $y_i \in \{0, 1, 2, \ldots\}$ vs the predicted intensity $\hat{\lambda}_i$.

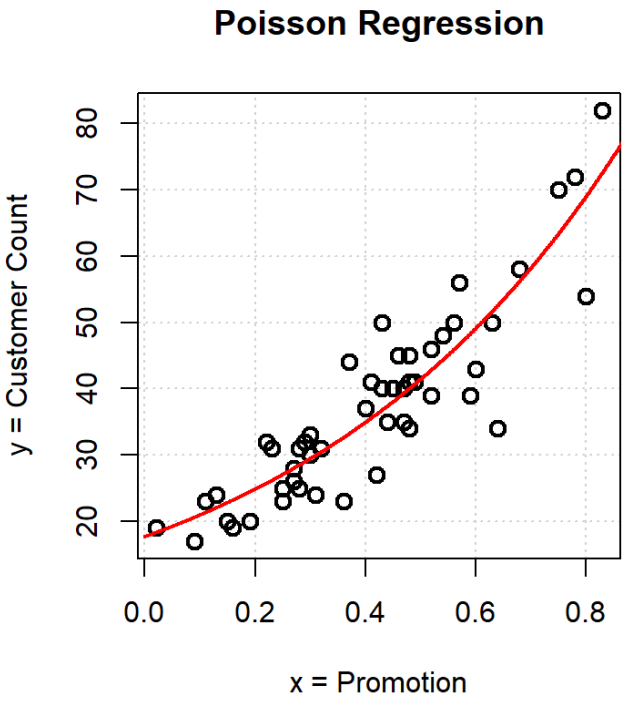
$$r_i^{\text{response}} = y_i - \hat{\lambda}_i$$

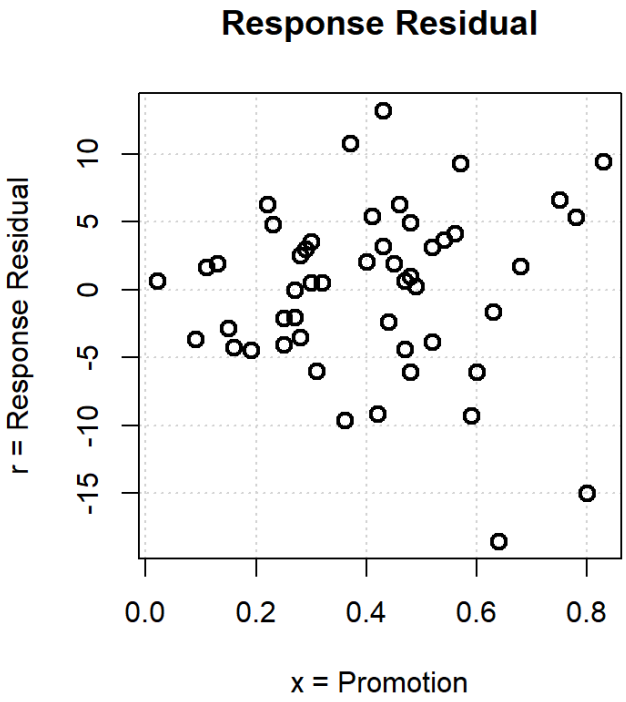
  - These residuals are not on the same scale of magnitude (i.e. not standardized nor have constant variance) across the data set because $\text{Var}(Y_i) = \lambda_i$ increases with the intensity λ_i itself.

```
# Response residual
lambda <- fitted(model)

y - lambda
residuals(model, type='response') # See ?residuals.glm
```

```
        1         2         3         4         5         6 
0.6843192 -3.6300077  1.6565778  1.9184925 -2.8451168 -4.2367676 
        1         2         3         4         5         6 
0.6843192 -3.6300077  1.6565778  1.9184925 -2.8451168 -4.2367676 
```



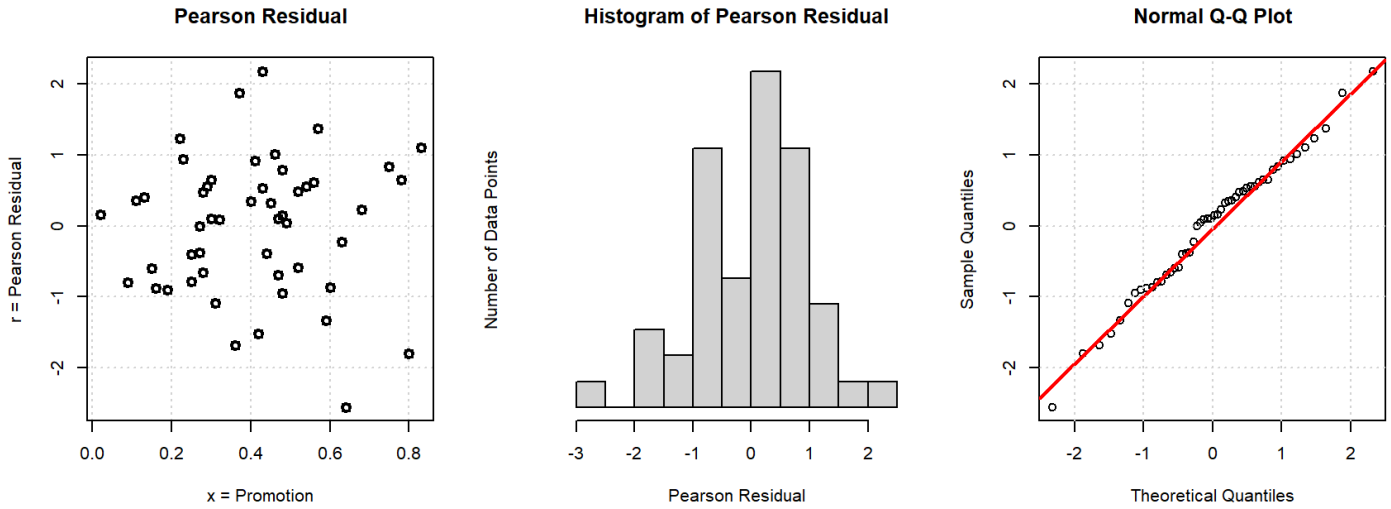


- **Pearson residual**: Attempt to standardize the response residual by dividing by its (estimated) standard error using $\text{Var}(Y_i) = \lambda_i \approx \hat{\lambda}_i$.

$$r_i^{\text{pearson}} = \frac{y_i - \hat{\lambda}_i}{\sqrt{\hat{\lambda}_i}}$$

```
# Pearson residual
residuals(model, type='pearson')
(y - lambda) / sqrt(lambda)
```

```
        1         2         3         4         5         6 
 0.1598995 -0.7992044  0.3585748  0.4082684 -0.5952555 -0.8789148 
        1         2         3         4         5         6 
 0.1598995 -0.7992044  0.3585748  0.4082684 -0.5952555 -0.8789148 
```



- **Deviance residual**: Inspired by the residual deviance formula

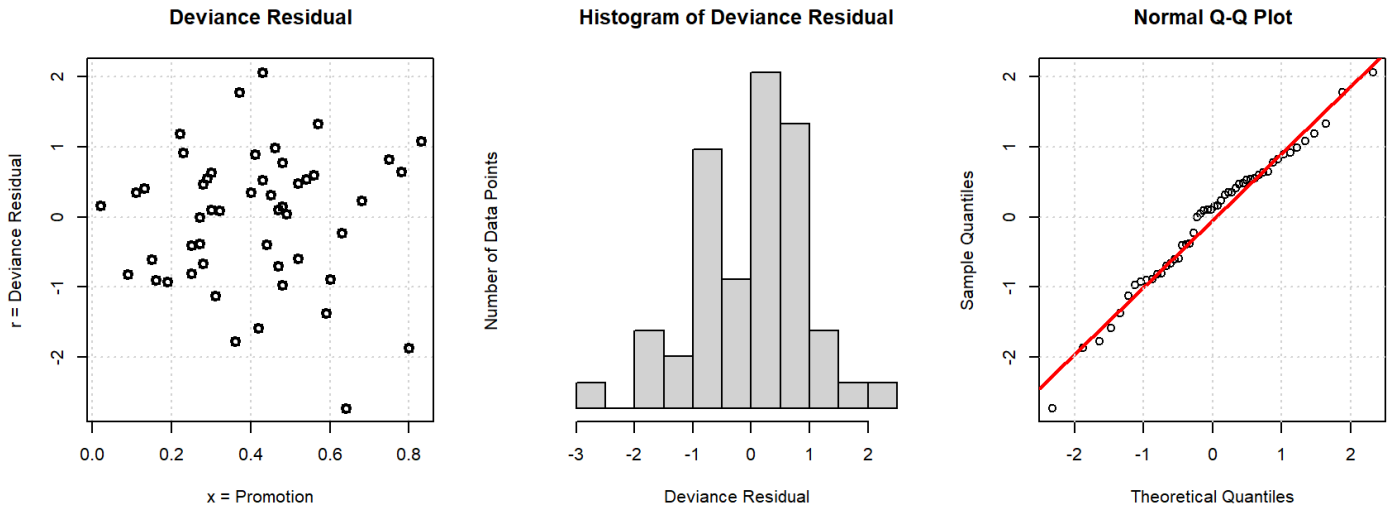
$$D_{\text{residual}} = \sum_{i=1}^{n} \underbrace{2\left(-(y_i - \hat{\lambda}_i) + y_i \log\left(y_i / \hat{\lambda}_i\right)\right)}_{\text{deviance residual ?}}$$

Let's define deviance residual for the ith data point to be

$$r_i^{\text{deviance}} = s_i \sqrt{2\left(-(y_i - \hat{\lambda}_i) + y_i \log\left(y_i / \hat{\lambda}_i\right)\right)} \qquad \text{where } s_i = \text{sign}(y_i - \hat{\lambda}_i)$$

```
# Deviance residual
residuals(model) # Default  Same as resid(model)
residuals(model, type='deviance')
summary(model)$deviance.resid
```

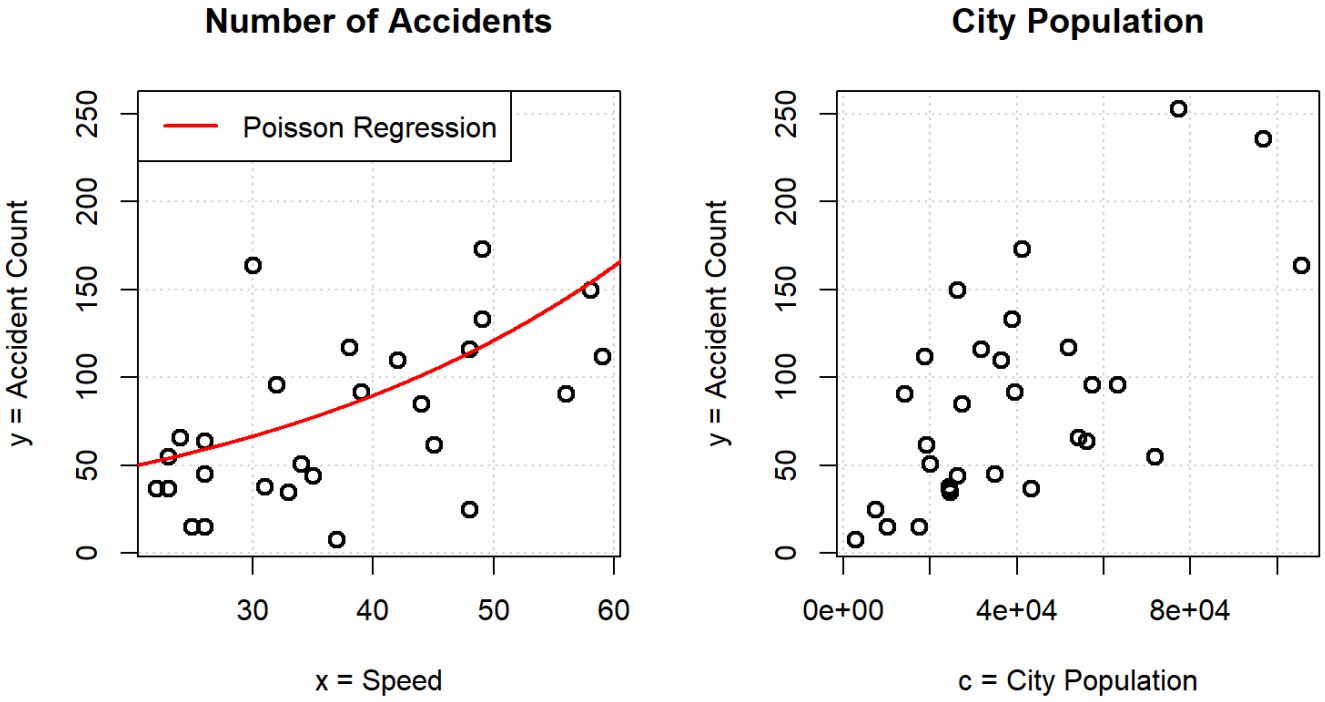
```
        1         2         3         4         5         6 
0.1589190 -0.8245491  0.3540802  0.4025608 -0.6083001 -0.9078847 
        1         2         3         4         5         6 
0.1589190 -0.8245491  0.3540802  0.4025608 -0.6083001 -0.9078847 
        1         2         3         4         5         6 
0.1589190 -0.8245491  0.3540802  0.4025608 -0.6083001 -0.9078847 
```



7.7 Rate Data and Offset

- *Example:* Let Y_i be the number of pedestrians in the ith city involved in traffic accident in a year. Let x_i be the average vehicle speed limit of the city and c_i be the population size of the city.

```
traffic <- data.frame(city=c(2698, 7344, 10029, 14111, 17432, 18677, 19168, 20072, 24407, 24428,
                             24509, 26191, 26245, 27300, 31796, 34888, 36349, 38826, 39503, 41199,
                             43153, 51919, 54076, 55974, 57233, 63082, 71706, 77107, 96637, 105508),
                      speed=c(37, 48, 26, 56, 25, 59, 45, 34, 23, 31,
                              33, 35, 58, 44, 48, 26, 42, 49, 39, 49,
                              22, 38, 24, 26, 32, 32, 23, 44, 41, 30),
                      accident=c(8, 25, 15, 91, 15, 112, 62, 51, 37, 38,
                                 35, 44, 150, 85, 116, 45, 110, 133, 92,
                                 173, 37, 117, 66, 64, 96, 96, 55, 253, 236, 164))
```




```
model <- glm(accident ~ speed, family = poisson, data = traffic)

summary(model)
pchisq(deviance(model), df.residual(model), lower.tail=F) # X^2 goodness-of-fit test
```

```
Call:
glm(formula = accident ~ speed, family = poisson, data = traffic)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) 3.298721   0.074632   44.20   <2e-16 ***
speed       0.029967   0.001761   17.02   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

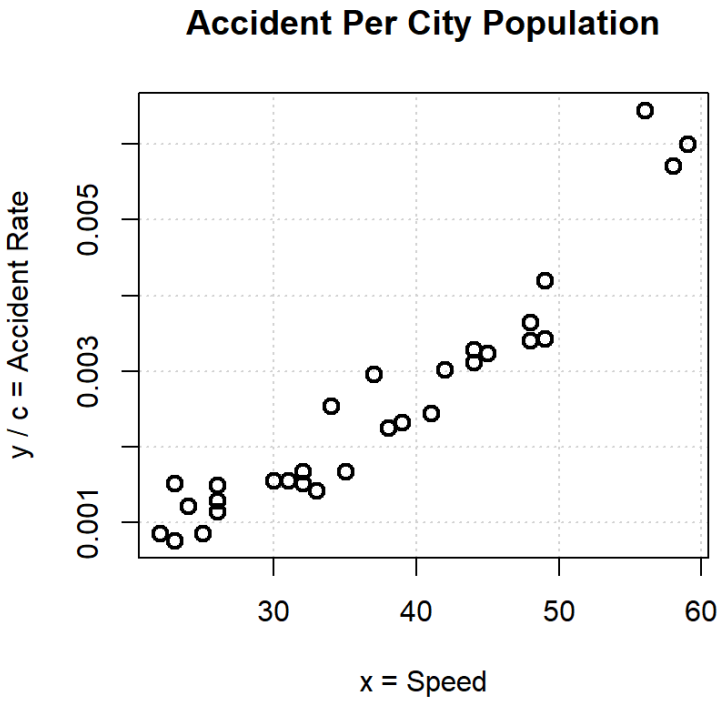
(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 1206.52  on 29  degrees of freedom
Residual deviance:  919.76  on 28  degrees of freedom
AIC: 1104.8

Number of Fisher Scoring iterations: 5

[1] 1.2829e-175
```

- Intuitively, a larger city with more population c_i might have more accidents y_i. (In statistics literature, c_i is called the **exposure** i.e the number of potential occurrences of the event.) Speed limit x_i is likely more related to the **rate** of accident y_i/c_i instead of the count of accident y_i itself.



- Let's consider the model equation with log link function as follows:

$$\mathbb{E}(Y_i) = \underbrace{\lambda_i}_{\text{intensity}} = \underbrace{c_i}_{\text{exposure}} \times \underbrace{e^{\beta_0+\beta_1 x_i}}_{\text{rate of occurrence}}$$

  The corresponding Poisson regression model is

$$Y_i \sim \text{Poisson}\left(\lambda_i = c_i e^{\beta_0+\beta_1 x_i}\right) \quad \text{for } i = 1, \ldots, n$$

Rewriting the model equation as

$$\log \mathbb{E}(Y_i) = \underbrace{\log(c_i)}_{\text{constant offset}} + \underbrace{\beta_0}_{\text{intercept}} + \beta_1 x_i$$

- The constant **offset** term (which is typically the logarithm of exposure) is part of the observed data points and not another unknown model intercept parameter (such as β_0) to be estimated.
- In R, use `glm(..., offset = ..., family=poisson)` or `glm(y ~ offset(...) + ..., family=poisson)` to specify the offset.

```
model.o <- glm(accident ~ speed, offset = log(city), family = poisson, data = traffic)
# Same as glm(accident ~ offset(log(city)) + speed, family = poisson, data = traffic)

# Offset
model.o$offset

summary(model.o)
pchisq(deviance(model.o), df.residual(model.o), lower.tail=F) # X^2 goodness-of-fit test
```

```
[1] 7.900266 8.901639 9.213236 9.554710 9.766063 9.835048

Call:
glm(formula = accident ~ speed, family = poisson, data = traffic,
    offset = log(city))

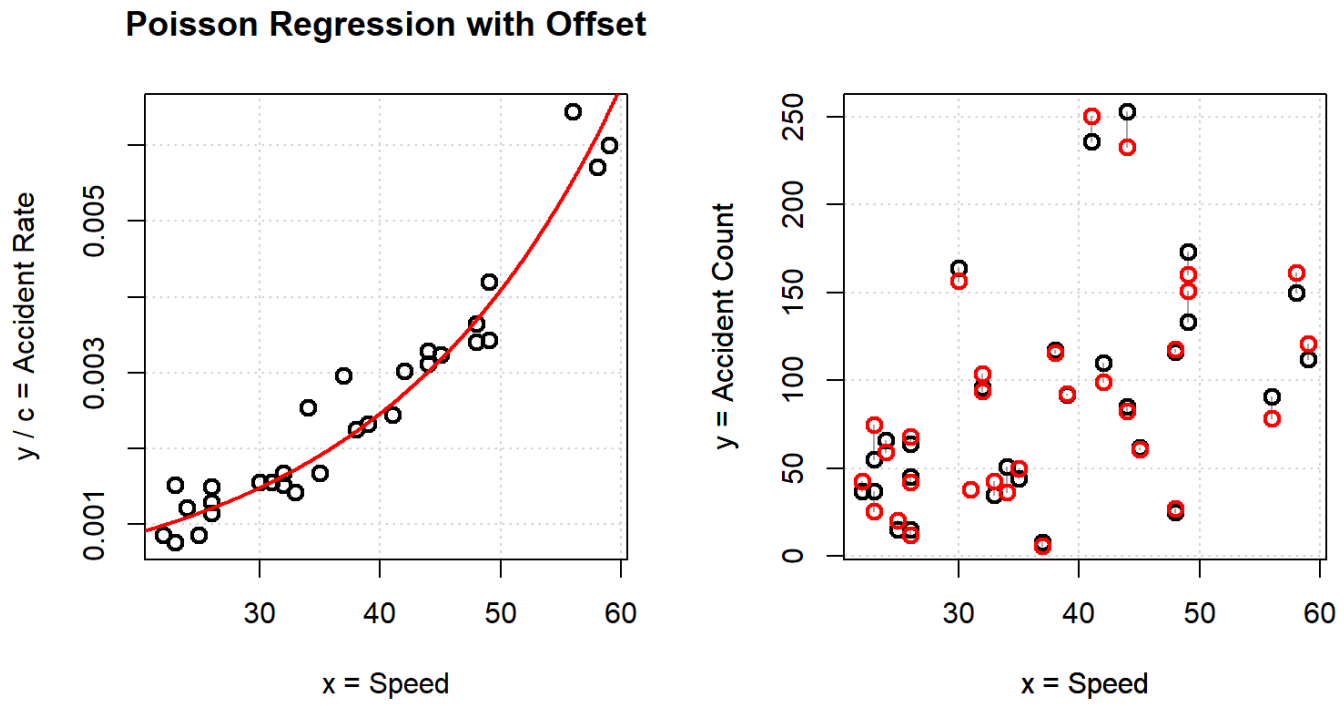
Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -8.035130   0.079980 -100.46   <2e-16 ***
speed        0.050731   0.001896   26.76   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 739.074  on 29  degrees of freedom
Residual deviance:  33.975  on 28  degrees of freedom
AIC: 219.05

Number of Fisher Scoring iterations: 4

[1] 0.201698
```



- If we do not identify the `offset()` explicitly in R's `glm()`, then $\log c_i$ is treated as just another explanatory variable x_{i2} with its own model parameter β_2 to be estimated. The model equation becomes

$$\mathbb{E}(Y_i) = c_i^{\beta_2} \times e^{\beta_0 + \beta_1 x_i}$$

```
model.c <- glm(accident ~ speed + log(city), family = poisson, data = traffic)

summary(model.c)
```

```

Call:
glm(formula = accident ~ speed + log(city), family = poisson,
    data = traffic)

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -7.721881   0.419399  -18.41   <2e-16 ***
speed        0.050079   0.002075   24.14   <2e-16 ***
log(city)    0.973113   0.035363   27.52   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 1206.525  on 29  degrees of freedom
Residual deviance:   33.399  on 27  degrees of freedom
AIC: 220.47

Number of Fisher Scoring iterations: 4
```

- If $\beta_2 \neq 1$, then it means the number of accidents do not increase exactly proportional to the city's population size. Let's do a Wald z-test to decide.

$$H_0 : \beta_2 = 1 \qquad \text{vs} \qquad H_a : \beta_2 \neq 1$$

The z-test statistic is

$$z_2 = \frac{\hat{\beta}_2 - 1}{\text{s.e.}(\hat{\beta}_2)}$$

```
# Step 1: Extract standard error for b2
C <- vcov(model.c) # Same as summary(model.c)$cov.scaled and $cov.unscaled
(s.e.b2 <- sqrt(diag(C)[3]))

# Step 2: Compute z-statistic for H0: b2 = 1
b2 <- coef(model.c)[3]
(z.stat <- (b2 - 1) / s.e.b2)

# Step 3: Compute (two-sided) p-value using reference N(0,1) distribution
(p.value <- 2 * pnorm(abs(z.stat), lower.tail = FALSE)) # Large p-value so cannot reject H0
```

```
 log(city)
0.03536323
 log(city)
-0.7603051
log(city)
0.4470722
```